



26021 RTOS5 - Mastering USB communication: Develop USB Device applications with Zephyr's USB Device Stack on your custom board

Introduction

USB is now a standard serial communication channel to connect embedded systems to PCs. Upon completion of the course, you will be able to create an enumerable USB device and transfer data between a Microchip MCU and a PC using Zephyr's USB Device Stack. The hands-on part of this class is based on a development board that is not yet supported by Zephyr tree, and you will also learn how to add your own board to a Zephyr based application.

Upon completion, you will be able to:

- Create your custom board support and compile the Zephyr blinky example on it.
- Add USB CDC ACM class to the example and communicate with the PC using a terminal.
- Transfer generic data between MCU and PC applications.

This class includes four labs:

Lab 1 - Run the Blinky sample project on your custom board

Lab 2 - Redirect the Console to the USB

Lab 3 - Build a CDC device and communicate with a terminal on PC

Lab 4 - Implement a simple command-response protocol

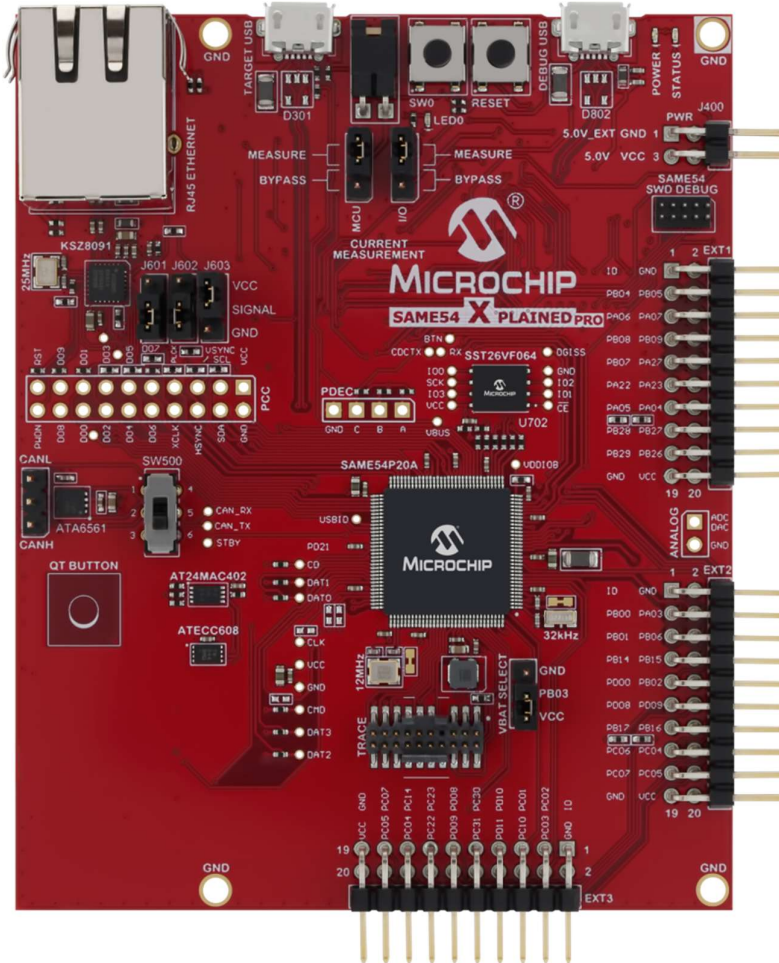
Prerequisites

The lab material assumes you have prior experience with:

- Zephyr RTOS environment
- C language programming

Microchip Development Tool Requirements

ATSAME54-XPRO - SAME54 Xplained Pro



2 x CABLE, USB A TO MICRO-B, 6 ft



Lab 1 - Run the Blinky sample project on your custom board

Purpose

In this lab, you will learn how to create Zephyr support for a custom board and run the Zephyr Blinky example on it.

Overview

In this lab, you will create board support for a custom hardware platform in Zephyr, focusing on defining the required Devicetree, board configuration files, and build system integration.

You will then build and flash the Zephyr *Blinky* example to verify your setup. The application uses GPIO to control an LED and outputs messages over UART.

Upon successful completion, the LED on your board should blink and you should see serial output in a terminal, confirming that your Devicetree, UART configuration, and board support are correctly implemented.

Procedure

The Beginning of your Lab Procedure. To add steps to your procedure use the step and substep macros.

You can copy the Blinky Sample code in Lab folder using the provided batch file:

```
C:\MASTERS\26021_RTOS5\Copy_Blinky_Sample.bat
```

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always .\Lab\ -- -DBOARD_ROOT="."
```

Then you can deploy the executable using **west** again:

```
west flash
```

In the **Serial Monitor** Extension, open the **Port** corresponding to the **EDBG** serial bridge at **115200** baud rate.

Results

Here is where your results go.

Summary

Here is where your summary goes.

Lab 2 - Redirect the Console to the USB

Purpose

In the previous lab, you viewed the serial output of your device through the USB bridge provided by the onboard programmer/debugger (**EDBG**).

In this lab you will learn how to move the communication from **UART** (SERCOM1) to **USB**

Overview

After redirecting the serial communication to **USB**, you will use the **Console** service, provided by **Zephyr OS**, to toggle **LED0**, if you receive the “**T**” character from the **Serial Monitor** Extension of **Visual Studio Code**, otherwise you will send back a message with the echo of the received character; if **SW0** on the board is pressed, you will send the string “**Button pressed!\n**” to the **Serial Monitor** (via **Console** service).

Procedure

This lab will continue from the previous lab. If you haven’t finished the previous lab, please overwrite the content of **board** and **Lab** folders with the **Solution/boards** and **Solution/lab1** folders.

You can copy the solution of Lab 1 over Lab and boards folders using the provided batch file:

```
C:\MASTERS\26021_RTOS5\COPY_Lab1_Solution.bat
```

or just run the following command from the Visual Studio Code terminal:

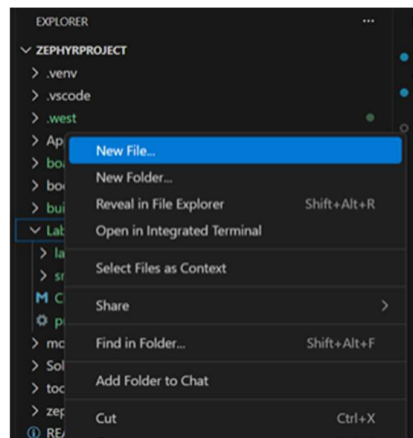
```
..\Copy_Lab1_Solution.bat
```

First, you need to redirect the Console from **SERCOM2** to **USB** and configure the USB node to work as **USB CDC-ACM** Device.

Instead of modifying the custom board Device Tree, you can use the Device Tree Overlay in your application.

Create a new **app.overlay** file in the **Lab** folder.

You can do that from the **Explorer** window right-clicking the **Lab** folder and selecting **New File...**



1. Step 1 – HINT

The property you need to overlay with **/chosen** node is **zephyr, console** and you need to set it to **&cdc_acm_uart**

You can use the **zephyr_udc0** node label to overlay the configuration of USB peripheral adding a child node labeled **cdc_acm_uart** with **compatible** property set to “**zephyr, cdc-acm-uart**”

Documentation is available here:

<https://docs.zephyrproject.org/4.3.0/build/dts/intro-syntax-structure.html>

<https://docs.zephyrproject.org/latest/build/dts/api/bindings/serial/zephyr%2Ccdc-acm-uart.html>

Next you need to enable software support for the **USB Device Stack**, the **USB CDC-ACM Class**, the **Console** and the **Input** services; you can do that by adding the proper Kconfig options to the **prj.conf** file.

2. Step 2 – HINT

The **Kconfig options** that you need to add are:

CONFIG_USB_DEVICE_STACK_NEXT: adds the USB Device Stack support

CONFIG_USBD_CDC_ACM_CLASS: adds the USB CDC-ACM Class support

CONFIG_CDC_ACM_SERIAL_INITIALIZE_AT_BOOT: initialize the stack at boot

CONFIG_CDC_ACM_SERIAL_ENABLE_AT_BOOT: enable the stack at boot

CONFIG_CDC_ACM_SERIAL_VID: assign the Vendor ID

CONFIG_CDC_ACM_SERIAL_PID: assign the Product ID

CONFIG_CDC_ACM_SERIAL_MAX_POWER: assign the max current drained by the device

(Value = max current / 2mA)

CONFIG_CDC_ACM_SERIAL_MANUFACTURER_STRING: assign the Manufacturer String

CONFIG_CDC_ACM_SERIAL_PRODUCT_STRING: assign the Product String

CONFIG_CONSOLE_SUBSYS: adds Console service support

CONFIG_CONSOLE_GETCHAR: enable character by character input and output

CONFIG_INPUT: add Input service support

Documentation is available here:

<https://docs.zephyrproject.org/4.3.0/kconfig.html>

Now you need to modify the example code.

Open **main.c** file and locate the **#include** directives at the beginning of the file; unlike the Blinky demo, you don't need to include the header file of **stdio**, but you need to include the header files of **Console** (zephyr/console/console.h) and **Input** (zephyr/input/input.h).

In this second lab, you are going to use the reference **button0 (SW0)**.

3. Step 3 - HINT

For the **button0 (SW0)**, you need to get the code property of the button and you can define it as **SW0_CODE** using the **DT_PROP** macro and the **DT_NODELABEL** macro to refer to **SW0 (button0)**

Documentation is available here:

<https://docs.zephyrproject.org/latest/build/dts/api/api.html#node-identifiers-and-helpers>

You are using the Input service that receive events from the **gpio-keys driver** and dispatch them. You can define a callback function, that prints the desired string when you press the **SW0** button, and register that function in the **Input** subsystem.

4. Step 4 – HINT

You can define a function named **input_cb()** that you will register in the **Input** service using the **INPUT_CALLBACK_DEFINE** macro. You can use **NULL** for **_dev** and **_user_data** parameters because they are not used in this lab. Define the function immediately before the **main()** function.

The function needs to have a prototype like the following:

void input_cb(struct input_event *event, void *user_data);

In the **input_cb** function, you need to check for the **event->code** and if it is related to **SW0** button (**SW0_CODE**) then you need to check if the event value is pressed (**event->value == true**); if both the condition are met, you can use the function **printk()** to print the **"Button pressed!\n"** string.

Documentation is available here:

<https://docs.zephyrproject.org/latest/services/input/index.html>

When you press a character key on the PC keyboard, you need to receive it and, based on the received character, you need to toggle **LED0** or print a message with the echo of the received character.

To receive a character, you can use the **Console** service that needs to be initialized. You can use the **console_init()** function to initialize the **Console** service.

5. Step 5 – HINT

You need to call **console_init()** function before invoking other **Console** APIs. You can call it in the **main()** function, before the **while(1)** loop.

console_init() function returns 0 on success or an error code (<0) otherwise. It is a good practice to check the returned value and do not continue if there is an error (**return 0**)

You can now print a string to let the user know what the application does

(e.g. **printk("\nPress T on the keyboard to toggle the LED 0\n - or - \nPress SW0 on the board to receive a message\n")**)

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_console_api.html

Inside the **while(1)** loop of the **main()** function, there is the code of the Blinky example.

You can reuse it, commenting-out the **k_msleep()** function call because in this lab we do not need the 1 second delay anymore; the remaining code inside the loop is what you need to do if the received character is **"T"**.

Now modify the code to receive a character, using the Console API, and toggle **LED0**, if you received the character **"T"**, or send back a string stating which character has been received.

6. Step 6 – HINT

You can use **console_getchar()** function to receive a character. You can assign the returned character in an unsigned char variable (**chr**), then test the value of that variable: if it is **"T"** you can use the Blinky code to toggle **LED0** and print the LED state (remember to comment-out the delay to avoid low responsiveness), otherwise you

need to send back a string echoing the received character using ***printk("Received character: %c\n", chr)***

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__console__api.html

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always .\Lab\ -- -DBOARD_ROOT=".."
```

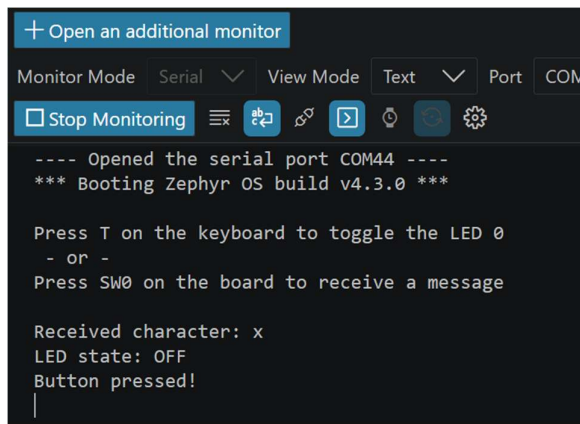
Then you can deploy the executable using **west** again:

```
west flash
```

Connect the second USB cable, connected to **"TARGET USB"** port on the board, to an available USB port of the PC and use the **Serial Monitor** Extension, opening the **Port** corresponding to the **USB CDC-ACM** Device at **115200** Baude rate.

Results

If you press **"x"**, then **"T"** and then the button **SW0**, you should see something like the following screenshot:



```
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM
[Stop Monitoring] [Refresh] [Connect] [Disconnect] [Settings]
---- Opened the serial port COM44 ----
*** Booting Zephyr OS build v4.3.0 ***

Press T on the keyboard to toggle the LED 0
- or -
Press SW0 on the board to receive a message

Received character: x
LED state: OFF
Button pressed!
|
```

Summary

Congratulations! You have successfully redirected the **Console** to **USB** and communicated with the **Serial Monitor** running on the PC!

This lab demonstrated how it is easy to “just” add USB communication, redirecting the **Console** service.

Extra Question

Have you noticed a strange behavior? The string you are sending after enabling the **Console** is not displayed in the **Serial Monitor** window until you press a key or the **SW2** button on the board. Can you guess why?

You can use **MPLAB AI Coding Assistant** to search for a possible reason...

Lab 3 - Build a CDC device and communicate with a terminal on PC

Purpose

In this lab you will learn how to configure the USB Device Stack to implement a USB CDC-ACM Device

Overview

Instead of just redirecting the Console, you will configure the USB Device Stack from scratch, creating the USB Device Controller context structure, defining the USB Device strings descriptors, the USB Configuration descriptors, the USB message notification callback function and, finally, you will initialize and enable the USB Device Stack.

Once enabled, wait for the Line Control signal, that indicate the PC terminal opened the communication with the USB device, before sending the initial string.

You will use the interrupt-driven API to send and receive data, like you did in the previous lab.

Procedure

This lab will continue from the previous lab. If you haven't finished the previous lab, please overwrite the content of **Lab** folder with the one you can find in the **Solution/lab2** folder.

You can copy the solution of Lab 2 over Lab folder using the provided batch file:

```
C:\MASTERS\26021_RTOS5\Copy_Lab2_Solution.bat
```

or just run the following command from the Visual Studio Code terminal:

```
..\Copy_Lab2_Solution.bat
```

First, you need to remove the redirection of the **Console** from the USB peripheral, removing the **/chosen** node you added in the previous lab from the **app.overlay** file.

You are going to configure the **USB Device Stack** in your code so you need to remove the **CDC_ACM_SERIAL** configuration options from the **prj.config** file and remove the **Console** configuration options as well, because you will not use the **Console API** in this lab. Finally, you need to add the configuration options for the **Serial** and **Hardware Information** drivers to use the **UART API** for the **Virtual CDC ACM UART** and enable the Serial Number string for the **USB CDC_ACM Device**.

- **Step 1 – HINT**

The **Kconfig options** that you need to add are:

CONFIG_SERIAL: enables the Serial drivers support

CONFIG_UART_INTERRUPT_DRIVEN: adds interrupt support for **UART**

CONFIG_UART_LINE_CTRL: enables the API to control the serial line

CONFIG_HWINFO: enable the **Hardware Information driver**

Documentation is available here:

<https://docs.zephyrproject.org/4.3.0/kconfig.html>

To improve the code readability, you can add the **USB Device Stack** configuration in separate files. Create two files, **usb.c** and **usb.h**, in the **Lab/src** folder.

In this lab you will use the **USB Device Controller APIs** to implement a Virtual UART so, in **usb.h** file, you need to include the header file of the USB module (`zephyr/usb/usbd.h`) and the one for the UART (`zephyr/drivers/uart.h`) because you will use UART API to send and receive data through the Virtual UART; then, including **usb.h** in the **usb.c** file, you can call the provided APIs.

You will need the device structure related to the **USB CDC-ACM Device** instance to interact with the **USB CDC-ACM Device** using the **UART Interrupt-driven APIs** and you can define it in the **usb.c** file.

- **Step 2 – HINT**

You can use the **DEVICE_DT_GET** macro to get a reference to the CDC interface that has **cdc_acm_uart** label in the **app.overlay** file; **DEVICE_DT_GET** requires a node identifier that you can retrieve from the label using **DT_NODELABEL** macro.

In **usb.c** file, define a device constant structure named **cdc_dev** and assign to it the reference to the **cdc_acm_uart** interface.

In **usb.h** file, declare **cdc_dev** as external, so you will be able to use it in the **main.c** file
Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_device_model.html

https://docs.zephyrproject.org/latest/doxygen/html/group_devicetree-generic-id.html

The USB Device Stack requires a context structure that you can define in the **usb.c** file

- **Step 3 – HINT**

You can use the **USBD_DEVICE_DEFINE** macro to create a context structure to initialize the USB Device stack.

Use the macro to create the structure named **usbd_ctx**, assigning **0x04D8** as **Vendor ID** and **0x000A** as **Product ID**; you can get a reference to the device structure of the **USB Device Controller**, labeled **zephyr_udc0**, using the two macros **DEVICE_DT_GET** and **DT_NODELABEL** like you have done in the previous step.
Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_devicetree-generic-id.html

https://docs.zephyrproject.org/latest/doxygen/html/group_usbd_api.html

You can use USB Device Stack macros to define the USB Device String Descriptors.

- **Step 4 – HINT**

You can use the following macros:

USBD_DESC_LANG_DEFINE: defines a descriptors node used as language string descriptor zero. Use it to define the *lang_desc* string descriptor

USBD_DESC_MANUFACTURER_DEFINE: defines a descriptors node used as manufacturer string descriptor. Use it to define the *mfr_desc* string descriptor initialized with *"Microchip Technology Inc."*

USBD_DESC_PRODUCT_DEFINE: defines a descriptors node used as product string descriptor. Use it to define the *prod_desc* string descriptor initialized with *"Zephyr CDC-ACM"*

USBD_DESC_SERIAL_NUMBER_DEFINE: defines a descriptors node used as serial number string descriptor. Use it to define the *sn_desc* string descriptor (it requires that **CONFIG_HWINFO** is enabled in *prj.conf* file).

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_usbd_api.html

Also, the configuration descriptor and the USB Configuration instance can be created using the provided macros.

- **Step 5 – HINT**

Use the macro **USBD_DESC_CONFIG_DEFINE** to create the *fs_cfg_desc* configuration descriptor, initialized with "FS Configuration", and the macro **USBD_CONFIGURATION_DEFINE** to create the *fs_config* configuration instance.

The second macro requires to declare if the device is a self or bus powered USB Device and the maximum current the device will drain from the USB Host: the value is a number that return the current value in 2mA steps ($250 * 2\text{mA} = 500\text{mA}$).

Declare your device as bus powered USB device (**USB_SCD_SELF_POWERED**) that

The *attrib* parameter has defines only for setting the flags so the parameter will be 0 for a bus powered USB device that does not implement Remote Wakeup (both flags set to zero)

drain 100mA

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_usbd_api.html

You should manage the USB Message notifications and you can define a callback function to handle them and register it into the USB Device Stack.

This time, you need to manually enable or disable the USB Device support based on the status of the Vbus pin: if it is active (cable connected) you need to enable USB Device support, otherwise (cable disconnected) you need to disable it.

- **Step 6 – HINT**

You can define a callback function and register it with the registration function provided by the **USB Device Core API**.

The prototype of the callback function is ***static void usb_msg_callback(struct usbd_context *const ctx, const struct usbd_msg *const msg);***

Define the callback function and enable or disable the **USB Device Stack** when the **USB Device Controller driver** can detect **VBUS** state changes: enable when the **type** member of the **msg** parameter is **USBD_MSG_VBUS_READY**, disable it when it is equal to **USBD_MSG_VBUS_REMOVED**

Like in the previous lab, you need to send an initial string explaining what the application does when connected so you will define, in a following step, a function that does that, with the following prototype:

void manage_usb_device(bool connected)

You can call that function to enable (passing **true**) or disable (passing **false**) the USB Device support.

You can check the **VBUS** Detection capability of the **USB Device Controller driver** with the function:

You will use this callback to manage the **USB Message notification** relevant for your application.

In this lab you only handle the messages related to a **UDC driver** that support **VBUS** detection. If the **UDC driver** does not provide that support, you need to use a **GPIO** to check when the cable is connected.

Writing the code to handle both cases, based on the capability of the **UDC driver**, makes your code more portable.

int usbd_can_detect_vbus(struct usbd_context *const uds_ctx)

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usb_api.html

Now you need to add USB String Descriptors, Configuration Descriptor and register the callback function in the USB Device Stack context structure. You can do that in a function that you can define in **usb.c** file and call it from **main.c** file. As mentioned in the previous step, you need handle also the case where the **UDC driver** does not detect Vbus, so you need to monitoring it in your code.

- **Step 7 – HINT**

In **usb.c** file, define a function named ***init_usb_device()***, without parameter, that returns an integer. Define a boolean variable, named ***managed_vbus***, to store the information about the Vbus detection capability of the **UDC driver**.

In **usb.h** file, declare the function prototype and the boolean variable as external to be used in the **main.c** file.

The **USB Device Core APIs** you will call to initialize the stack return an error value (0 == no error). In case of error, the ***init_usb_device()*** function will return the error code to the caller function.

You need to add the previously defined USB Device String Descriptors.

- **Step 8 – HINT**

You can use the function ***int usbd_add_descriptor(struct usbd_context *const uds_ctx, struct usbd_desc_node *const desc_nd)*** to add the strings to the ***usbd_ctx*** structure:

- ***lang_desc***
- ***mfr_desc***
- ***prod_desc***
- ***sn_desc***

Remember to assign the returned value to an integer variable (***err***) and return it in case of error.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usbd_api.html

Then, you need to add the configuration descriptor.

- **Step 9 – HINT**

You can add the configuration ***fs_config*** to ***usbd_ctx*** using ***int usbd_add_configuration(struct usbd_context *const uds_ctx, const enum usbd_speed speed, struct usbd_config_node *const cfg_nd)*** function.

The second parameter is the USB Speed and the **ATSAME54P20A** only support Full-Speed (**USBD_SPEED_FS**). Remember to return the eventual error (***err***).

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usbd_api.html

You also need to register the USB Class in the context structure.

- **Step 10 – HINT**

You can register the **USB CDC-ACM** instance in ***usbd_ctx*** using ***int usbd_register_class(struct usbd_context *const uds_ctx, const char *name, const enum usbd_speed speed, const uint8_t cfg)*** function.

The second parameter is a unique identification string used to link a specific USB class instance; the standard identification strings used in Zephyr are:

- ***hid_n***: Used for Human Interface Devices
- ***cdc_acm_n***: Used for virtual UART controllers
- ***msc_n***: Used for Mass Storage Class instances
- ***uac2_n***: Used for USB Audio Class 2 devices
- ***cdc_ecm_n***: Used for Ethernet device emulation

n is the instance number.

The third parameter is the USB Speed (**USBD_SPEED_FS**) and the forth is the configuration number (***1***). Remember to return the eventual error (***err***).

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usbd_api.html

Next, you need to register the USB Message notification callback function.

- **Step 11 – HINT**

You can register the ***usb_msg_callback()*** function in ***usbd_ctx*** using ***int usbd_msg_register_cb(struct usbd_context *const uds_ctx, const usbd_msg_cb_t cb)*** function. Remember to return the eventual error (***err***).

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Now the USB Device Stack configuration for a USB CDC-ACM Device is completed and the stack can be initialized.

- **Step 12 – HINT**

You can Initialize the stack using the ***int usbd_init(struct usbd_context *const uds_ctx)*** function and the configured ***usbd_ctx*** context. Remember to return the eventual error (***err***).

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Finally, you need to set the value of ***managed_vbus*** variable and return the eventual error.

- **Step 13 – HINT**

Like you did in Step 6, you can check the **VBUS** Detection capability of the **USB Device Controller** driver with the function:

int usbd_can_detect_vbus(struct usbd_context *const uds_ctx)

and assign the returned value to ***managed_vbus***; then you need to return the error (***err***) value.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

As already mentioned in **Step 6**, you are going to use a function to enable or disable the USB Device Stack.

- **Step 14 – HINT**

In ***usb.c*** file, define a void function named ***manage_usb_device()***, with a boolean parameter named ***connected***.

In ***usb.h*** file, declare the function prototype to be called from the ***main.c*** file.

In interrupt-driven mode, you need to fill in the **Virtual UART FIFO** and enable the transmit Interrupt, to send something to the host, while you get a receive interrupt when the host send you data that you need to extract from the **Virtual UART FIFO**. Remember that in an USB communication, only the host can initiate a transfer: sending something to the host means prepare the data and wait that the host asks for them.

You are going to use two data structures to store received data and data to send.

- **Step 15 – HINT**

In **usb.h**, define a structure type, named **vcp_buffer**, with two members: **buf** and **length**. The type of **length** should be an unsigned 16-bit integer while **buf** should be an array of unsigned 8-bit integer; the size of the array should be enough to contain the larger amount of data you are expecting to receive/transmit. You are going to send the initial message when the Serial Monitor opens the Virtual COM Port (VCP) associated with your board so declare an external variable named **data_to_send** of type **vcp_buffer**.

The typical size of buffers, in a USB application, is a multiple of the Endpoint size. In this lab you need to send a string of 99 characters and the Bulk Endpoint size for a Full-Speed USB Device is 64 bytes so you can use 128 as size for both buffers.

Inside the **manage_usb_device()** function, you need to check if the cable is connected, relying on the **connected** parameter, and enable or disable the USB Device support accordingly. You also need to correct the behavior of the previous lab, waiting the Serial Monitor to open the communication with the board, before sending the first message to it.

- **Step 16 – HINT**

If **connected** is **true**, it means cable is connected so you can enable the USB Device support with the function **int usbd_enable(struct usbd_context *const uds_ctx)** passing the pointer to the context structure you defined (**usbd_ctx**). Then you can wait the DTR signal defining a variable of type **uint32_t** named **dtr** where to store the line control value and initialize it to **0**; in a while loop, where you looping if **dtr** is still **0**, you can call the **uart_line_ctrl_get()** function to check for Data Terminal Ready (**UART_LINE_CTRL_DTR**) and wait for 100ms using **k_sleep()** function in combination with the **K_MSEC** macro. Then, when the DTR is not zero and the while loop ends, you can use **sprintf()** function to insert the initial strings, you were sending with **printf()** in the previous lab, in **data_to_send.buf** and assign the returned value to **data_to_send.length**. Finally, you enable TX interrupt with **void uart_irq_tx_enable(const struct device *dev)** function, where the **dev** parameter is the **Virtual UART** device instance (**cdc_dev**). If **connected** is **false**, it means cable is disconnected so you can disable the USB Device support with the function **int usbd_disable(struct usbd_context *const uds_ctx)** passing the pointer to the context structure you defined (**usbd_ctx**).

For simplicity, **manage_usb_device()** is a void function so you do not need to check the return values of the APIs for errors.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usbd_api.html

https://docs.zephyrproject.org/latest/doxygen/html/group_uart_interrupt.html

https://docs.zephyrproject.org/latest/doxygen/html/group_thread_apis.html

https://docs.zephyrproject.org/latest/doxygen/html/group_clock_apis.html

You have completed the sequence to initialize the USB Stack that you will actually enable calling the **`init_usb_device()`** function from your application code. In this lab, you are not going to use the **Console** anymore so the **`printf()`** and **`printk()`** functions need to be commented out because they are not going to be redirected to the **Virtual UART**. Moreover, you will use interrupt driven approach so the content of the **`while(1)`** loop in your **`main()`** function will be moved into the callback function that handle Interrupt request of the **Virtual UART**.

in **`main.c`**, comment-out the include directive for the **Console** (`zephyr/console/console.h`), the call to **`printk()`** in the **`input_cb()`** callback function and, in the **`main()`** function, the calls to **`console_init()`**, **`printk()`** and the content of the **`while(1)`** loop. Comment-out also the declaration of the local variable **`led_state`** in the **`main()`** function to avoid the unused variable warning.

For simplicity, in this class we are using a single application thread and, having commented-out the blocking call to **`console_getchar()`** from the **`while(1)`** loop, you need to re-enable the call to **`k_msleep()`** function to allow the **Input service** to work properly.

Remove the comment from **`k_msleep(SLEEP_TIME_MS)`** in the **`while(1)`** loop to re-enable it.

Now you have removed the **Console** so you need to add the code to use the **Virtual UART**. To call **UART API** and the functions you defined in previous steps, you need to include the corresponding header files.

- **Step 17 – HINT**

Locate the last include directive in **`main.c`** file and include the header file you created in previous steps.

In **Step 15** you defined the structure type of the buffer you will use to send and receive data and declared the structure **`data_to_send`** as external to be used in both **`usb.c`** and **`main.c`** files.

You need to define the two data structures before using them.

- **Step 18 – HINT**

Before **`input_cb()`** callback function, define two data structures, **`received_data`** and **`data_to_send`**, of type **`vcp_buffer`**

The strings you were sending with **`printk()`** function inside the **`input_cb()`** function, now require you to place them into **`data_to_send`** buffer and enable TX interrupt to be sent.

- **Step 19 – HINT**

You can use **`sprintf()`** function to insert the string, you were sending with **`printk()`** previously, in **`data_to_send.buf`** and assign the returned value to **`data_to_send.length`**.

You can then enable TX interrupt with **`void uart_irq_tx_enable(const struct device *dev)`** function, where the `dev` parameter is the **Virtual UART** device instance (**`cdc_dev`**) Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_uart_interrupt.html

You need to handle the TX and RX IRQs related to the Virtual UART (USB peripheral) and you can register a callback function that it is called when there is an event (interrupt) related to it. In the callback function, while there is a pending interrupt, you need to handle the event.

- **Step 20 – HINT**

Define **`uart_irq_cb()`** callback function to handle the events, just before the **`main()`** function. The prototype of that function needs to be:

`void uart_irq_cb(const struct device *dev, void *user_data);`

Define , inside the function, a static boolean variable (**`led_state`**) that you will use to know the status of **LED0**, initializing it to true.

Add a **`while`** loop that run until the irq update function (**`int uart_irq_update(const struct device *dev)`**), that start processing interrupts in ISR, and irq pending function (**`int uart_irq_is_pending(const struct device *dev)`**) are both true.

You will fill the content of the **`while`** loop in the following steps.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_uart_interrupt.html

Inside the while loop, you need to check if data is ready and, if available, extract it from the Virtual UART FIFO. Once extracted, you need to check the size of the received data (**`>0`**) and the content of the first byte: if it is the character '**`T`**', you need to toggle the LED, update the **`led_state`** value and print the status of the LED; otherwise you need to send a string with the echo of the received character. Remember to enable the TX interrupt.

- **Step 21 – HINT**

You can use the **`uart_irq_rx_ready()`** function to check if something has arrived; then you can use **`uart_fifo_read()`** function to extract the data from the FIFO and copy them into **`received_data.buf`**, updating the size of received data, assign it to **`received_data.length`**.

If length is higher than zero, you need to check the content of **`received_data.buf[0]`** and if it is '**`T`**', you can toggle the **LED** using **`gpio_pin_toggle_dt()`** function, then invert the **`led_state`** value, prepare a string, with **`sprintf()`** function, to inform the host on the status of the **LED**; if it is not the character '**`T`**', prepare a string to echoing the received character. After preparing the strings, you need to enable the TX interrupt.

You can reuse the code you commented-out in the while(1) of the main function as reference.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_uart_interrupt.html

https://docs.zephyrproject.org/latest/doxygen/html/group_gpio_interface.html

You also need to check if the device is ready to send something and fill-in the FIFO, if ready. After filling it, you need to disable the TX interrupt because the event TX complete is not implemented yet and, if you do not disable the interrupt, at soon as the FIFO is sent to the host, the device is ready and the same interrupt is triggered, refilling the FIFO with te same string, sending it again and again.

- **Step 22 – HINT**

You can use the **`uart_irq_tx_ready()`** function to check if the Virtual UART is ready to accept additional data to be sent to the host, then you can use **`uart_fifo_fill()`** function to copy the data from the buffer **`data_to_send.buf`** to the FIFO; you can use **`data_to_send.length`** as size parameter.

Finally you need to disable the TX interrupt with **`uart_irq_tx_disable()`** function. Documentation is available here:

`uart_irq_tx_ready()` function returns the number of bytes you can add to the FIFO while **`uart_fifo_fill()`** function returns the actual number of bytes that were added to the FIFO, from the buffer.

In this lab we are not checking the return value of **`uart_fifo_fill()`**, but in your actual application it is a good practice to doublecheck if the number of bytes written is equal to the number of bytes you wanted.

https://docs.zephyrproject.org/latest/doxygen/html/group_uart_interrupt.html

In **Step 19**, you removed the **`printk()`** call and substituted it with a call to **`sprintf()`** function, followed by the TX interrupt enable function to start the transmission. Actually, no communication will take place until you initialize the USB Device Stack and register the Virtual UART callback function.

In the **`main()`** function, go just before the **`while(1)`** loop and add the code related to the following steps immediately it.

First, you need to check if the device is ready for use and the device pointer is a valid handle, then you can call the initialization routine you defined in **`usb.c`** file.

- **Step 23 – HINT**

You can use the **`device_is_ready()`** function to verify if the device pointed by **`cdc_dev`** is ready; if not, you can return **`0`** to the kernel (ending the main thread). If the handle is valid, you can call the **`init_usb_device()`** function you previously defined and check the returned value: if it is **`<0`** then return **`0`** to the kernel.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_device_model.htm

Then, you need to register the **`uart_irq_cb()`** function and enable the RX interrupt.

- **Step 24 – HINT**

You can use the **`uart_irq_callback_set()`** function to register **`uart_irq_cb()`** for the Virtual UART associated with **`cdc_dev`** handle, and **`uart_irq_rx_enable()`** to enable its RX interrupt.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_uart_interrupt.html

In **Step 6** you defined the USB Message Notification callback function where you enable or disable the USB Device support based on the notification message coming from the **`UDC`**

driver.

As mentioned, if the UDC driver does not monitor the presence of the Vbus, you need to use a GPIO pin to enable or disable the USB Device support.

In the first lab you defined the **vbus0** pin that is active when the Vbus is present. You need to call the **manage_usb_device()** function when there is a transition from inactive to active and vice-versa, so you can use the input service, like you did for button0, but only if the **UDC driver** does not monitor Vbus.

- **Step 25 – HINT**

You need to get the code property of the Vbus pin and you can define it as **VBUS_CODE** using the **DT_PROP** macro and the **DT_NODELABEL** macro to refer to Vbus pin (**vbus0**). You need to define it before the **input_cb()** function.

In the **input_cb()**, you can check if the **managed_vbus** variable is false and then check for the **event->code**: if it is related to Vbus pin (**VBUS_CODE**) then you need to check if the event value is active (**event->value == true**) and in that case enable the USB Device support using the **manage_usb_device()** function, passing **true**, otherwise disabling it, passing **false**.

Documentation is available here:

<https://docs.zephyrproject.org/latest/build/dts/api/api.html#node-identifiers-and-helpers>

The previous step does not cover the case where the USB cable is already connected at Power-On so you need to take care about that in your code.

- **Step 26 – HINT**

You need to define a container for GPIO information to be used with the GPIO APIs; the container has type **static const struct gpio_dt_spec** and you can call it **vbus_gpio**. You need to initialize it with the reference to **vbus0** pin that you can obtain with the macro **GPIO_DT_SPEC_GET()** where the node can be referenced using the **DT_NODELABEL()** macro and the property **gpios**.

Now, in the **main()** function, just before the **while(1)** loop, you can check if the Vbus does not manage the Vbus (**!managed_vbus**) and then, if the **vbus_gpio** is high, you can call the **manage_usb_device()** function, passing **true**. You can check the status of the GPIO pin with the function **int gpio_pin_get_dt(const struct gpio_dt_spec *spec)** that returns 1 if the pin is in active state or 0, if inactive.

Documentation is available here:

<https://docs.zephyrproject.org/latest/build/dts/api/api.html#node-identifiers-and-helpers>

https://docs.zephyrproject.org/latest/doxygen/html/group_gpio_interface.html

Finally, you added a new source file (**usb.c**) to the project so you need to update the list of target sources in **CMakeLists.txt** file.

- **Step 27 – HINT**

Add **src/usb.c** to the **target_source()** list.

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always .\Lab\ -- -DBOARD_ROOT=".."
```

Then you can deploy the executable using **west** again:

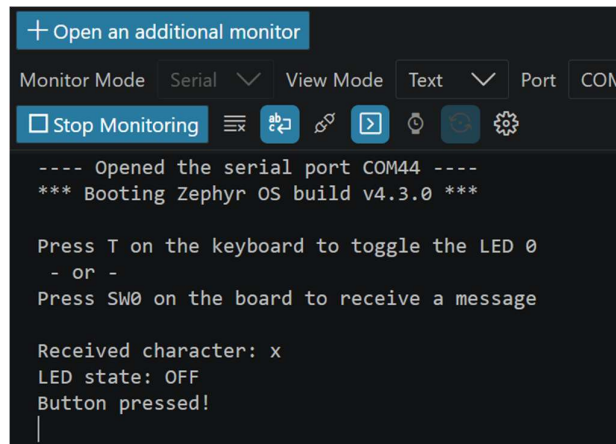
```
west flash
```

Connect the second USB cable, connected to **"TARGET USB"** port on the board, to an available USB port of the PC and use the **Serial Monitor** Extension, opening the **Port** corresponding to the **USB CDC-ACM** Device at **115200** Baude rate.

Results

You are going to have the same results as Lab 2, but this time you are using the USB CDC-ACM Device you created from scratch, not just redirecting the **Console**.

Like in Lab 2, if you press **"x"**, then **"T"** and then the button **SW0**, you should see something like the following screenshot:



```
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM
[ ] Stop Monitoring [ ] [ ] [ ] [ ] [ ] [ ]
---- Opened the serial port COM44 ----
*** Booting Zephyr OS build v4.3.0 ***

Press T on the keyboard to toggle the LED 0
- or -
Press SW0 on the board to receive a message

Received character: x
LED state: OFF
Button pressed!
|
```

Summary

Congratulations! You have successfully configured the USB Device Stack to implement a **USB CDC-ACM Device** and communicated with the **Serial Monitor** running on the PC!

Lab 4 - Implement a simple Command-Response protocol

Purpose

In this lab you will learn how to configure the USB Device Stack to implement a USB HID Device and how to implement a simple Command-Response protocol to send and receive generic data.

Overview

You will reconfigure the USB Device Stack to implement a USB HID Device, creating the HID Report Descriptor structure, defining the HID Device Operation structure, registering them in the USB device context, then you will define the HID Device Operation callback functions, implementing the Command-Response protocol to have your device communicate with the provided PC application.

Procedure

This lab will continue from the previous lab. If you haven't finished the previous lab, please overwrite the content of **Lab** folder with the one you can find in the **Solution/lab3** folder.

You can copy the solution of Lab 3 over Lab folder using the provided batch file:

```
C:\MASTERS\26021_RTOS5\Copy_Lab3_Solution.bat
```

or just run the following command from the Visual Studio Code terminal:

```
..\Copy_Lab3_Solution.bat
```

First, you need to reconfigure the USB node to work a **USB HID Device**.

1. Step 1 – HINT

Change the **zephyr_udc0** node label, commenting-out the **cdc_acm_uart** child node and adding a child node labeled **hid0** with **compatible** property set to "**zephyr, hid-device**", **protocol-code** property set to "**none**", **in-report-size** set to **64** as well as the **out-report-size**, then set both **in-polling-period-us** and **out-polling-period-us** to **1000** (1ms)

Documentation is available here:

<https://docs.zephyrproject.org/4.3.0/build/dts/intro-syntax-structure.html>

<https://docs.zephyrproject.org/latest/build/dts/api/bindings/usb/zephyr%2Chid-device.html>

Then you need to modify the **prj.conf** file to include support for the USB HID Class.

2. Step 2 – HINT

Comment-out the option related to CDC-ACM, Serial, UART and input and add the **kconfig option** to enable the support for the USB HID Class

(CONFIG_USBD_HID_SUPPORT)

Documentation is available here:

<https://docs.zephyrproject.org/4.3.0/kconfig.html>

The basic configurations for the USB Device Stack you developed in Lab 3 can be reused to implement the USB HID Device, with some modifications.

3. Step 3 – HINT

In the previous lab you defined a device handle for the USB CDC-ACM Device (***cdc_dev***). In this lab you need to change it to define a device handle for the USB HID Device (***hid_dev***).

You can use the node label ***hid0*** instead of ***cdc_acm_uart*** to initialize the handle.

Like you did in Lab 3, declare it as external in ***usb.h*** and define it in ***usb.c*** file.

Comment-out the include directives of **UART driver** (<zephyr/drivers/uart.h>) and include the header file to use the dedicated USB HID APIs in ***usb.h*** (zephyr/usb/class/usbd_hid.h)

The PC application provided for this lab is expecting a USB HID device with Product ID equal to 0x003F so you need to modify the definition of ***usbd_ctx*** accordingly.

4. Step 4 – HINT

Change the **PID** from ***0x000A*** to ***0x003F*** in the ***usbd_ctx*** definition.

To easily find your USB Device in the list of connected devices, it is better to change the Product String.

5. Step 5 – HINT

Change the ***prod_desc*** string from ***"Zephyr CDC-ACM"*** to ***"Zephyr vendor HID"***

You need to define a USB HID Report descriptor for the USB HID Device. The descriptor is an array of unsigned 8-bit integer that consists of specific items defined by the HID usage tables. An easy way to create that array is to ask **MPLAB AI Coding Assistant**.

6. Step 6 – HINT

Ask to the **MPLAB AI Coding Assistant** to generate for you a HID report descriptor, named ***hid_report_descriptor***, for a vendor HID Report of 64 elements of 1 byte each, used for sending and receiving data. Copy the array in your code before the ***init_usb_device()*** function.

Other than the USB Device Stack Notification Messages, in a USB HID Device you need to handle the USB HID User callbacks.

7. Step 7 – HINT

Declare ***hid_get_report_cb()***, ***hid_set_report_cb()***, ***hid_output_report_cb()*** and ***hid_input_report_done_cb()*** functions in ***usb.h*** file and define, in ***usb.c*** file, an ***hid_device_ops*** structure, named ***hid_ops***, assigning the functions to the corresponding members ***get_report***, ***set_report***, ***output_report*** and ***input_report_done***. Add it before ***init_usb_device()***.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/structhid_device_ops.html

You also need to modify the **init_usb_device()** function you defined in the previous lab to register the HID class, instead of the CDC-ACM Class, register the HID Report Descriptor and user callbacks, and set the code triple for an HID Device.

8. Step 8 – HINT

Comment-out the CDC-ACM Class registration and use **usbd_register_class()** function to register the HID Class (**hid_0**) then you can use **hid_device_register()** to register in **hid_dev** the defined **hid_report_descriptor**, with its size, and a pointer to the **hid_ops** structure. You can set the code triple of **usbd_ctx** to **[0, 0, 0]** with the **usbd_device_set_code_triple()** function.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usbd_hid_device.html

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usbd_api.html

To complete the reconfiguration of the USB Device Stack to enumerate the device as USB HID Device, you also need to comment-out, in **manage_usb_device()** function, the check for the **DTR**, the string definition and the TX interrupt enable functions.

Next steps are removing the old USB CDC-ACM related code in **main.c** and adding the HID related code.

Comment-out the code property definition **SW0_CODE** and the related code in the **input_cb()** function, the two structures you used to send and receive data (**received_data** and **data_to_send**), the UART IRQ callback (**uart_irq_cb**). In the **main()** function, comment-out the registration of the UART IRQ callback, the RX interrupt enable function; finally, you need to change the device handle you are checking in your code.

9. Step 9 – HINT

Replace the device handle in the **device_is_ready()** function with **hid_dev**.

In this lab the Input service is not used but the button is monitored as well.

10. Step 10 – HINT

Define, immediately before **main()** function, the container **button** of type **static const struct gpio_dt_spec**, initializing it using the macro **GPIO_DT_SPEC_GET()** with property **gpios** and using the macro **DT_NODELABEL()** to get the devicetree node identifier of **button0**.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_devicetree-generic-id.html

https://docs.zephyrproject.org/latest/doxygen/html/group_gpio_interface.html

You need to configure the corresponding GPIO to be used as an input pin so you can get its status (pressed/not pressed).

11. Step 11 – HINT

Check the readiness of the previously defined **button** using the **gpio_is_ready_dt()** function, returning **0** if it is not ready, then configure it to be a **GPIO_INPUT** with the **gpio_pin_configure_dt()** function, assigning the returned value to the variable **ret** and check if **ret** is less than zero, returning **0** if yes. The code needs to be placed in the **main()** function and you can put it immediately after the similar code for **led** container.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxxygen/html/group_gpio_interface.html

Now you need to define the user callbacks to handle the communication with the USB Host and implement the Command-Response protocol used by the PC application.

12. Step 12 – HINT

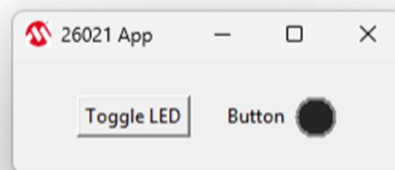
In this lab, the two required callback functions for **hid_get_report** and **hid_set_report** members of **hid_ops** are not used so you can define them as functions that just return **-ENOTSUP**. Define them before **main()** function.

-ENOTSUP indicate that the feature is not supported.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxxygen/html/structhid_device_ops.html

The PC application ("**26021 app.exe**") is a simple application to toggle a LED and show the current status of a button on the board.



The app looks for a USB HID Device with VIP = 0x04D8 and PID = 0x003F and, if the Device is connected to the Host, it enables the "Toggle LED" button. Pressing that button, the app sends an output report with the "Toggle LED" command, setting the first byte of the output report at **0x80**; no answer from the device is expected for this command.

The app also periodically send an output report with "Get button status" command, setting the first byte of output report at **0x81**, followed by a request of an input report, expecting a reply where the first byte of the report is the echo of the command (**0x81**) and the second byte is **0**, if the button is pressed, **1** if it is not pressed; the app change the color of the "Button" image based on the answer.

First, you need to define the callback that handles the output report.

13. Step 13 – HINT

Define the ***hid_output_report_cb()*** function you already declared in **usb.h** file. In the function, define two local **static** variables: a boolean ***led_state***, initialized at true, to store the status of the LED, and an unsigned 8-bit integer array of 64 bytes named ***input_report***, that will be used to provide the answer to the "Get button status" command.

Check if the size of the received buffer (***len***) is greater than 0 and if the pointer to the buffer (***buf***) is not pointing to NULL, then add a switch-case statement looking for the value of the first byte of the buffer: the two cases you need to implement are ***buf[0] = 0x80*** and ***buf[0] = 81***

The first command you need to implement is "Toggle LED" and you can use the ***led_state*** variable to report the new state of the LED via the serial bridge provided by the onboard debugger (like in Lab1).

14. Step 14 – HINT

You can toggle the LED using the ***gpio_pin_toggle_dt()*** function (returning immediately if the function does not return 0), then you need to negate the ***led_state*** variable to keep track of LED state.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_gpio_interface.html

Then you need to implement the "Get button status" command, where you need to prepare and send the response to the command.

15. Step 15 – HINT

You can use the ***input_report*** array to prepare the response, setting the first bite at 0x81 and the second byte at:

- **0**: SW2 is pressed
- **1**: SW2 is pressed

You can get the status of the SW2 button using the ***gpio_pin_get_dt()*** function that returns true if the pin is active (button pressed) or false if the pin is inactive (button not pressed).

Once the array is filled with the response, you can send it to the host submitting the request with ***hid_device_submit_report()*** function.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group_gpio_interface.html

https://docs.zephyrproject.org/4.3.0/doxygen/html/group_usb_hid_device.html

Last user callback you need to define is the one required for ***input_report_done*** member, that is called when the input report has been received by the host.

16. Step 16 – HINT

Define the ***hid_input_report_done_cb()*** function and use ***printk()*** to send the string containing the state of the LED (similar to the ***printf()*** you can find in the ***while(1)*** loop of the ***main()*** function); do the same to send the string containing the state of the button you reported with the input report (similar to the previous one but checking ***report[1]*** to decide if you need to print ***"Button is pressed\n"*** or ***"Button is not pressed\n"***).

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always .\Lab\ -- -DBOARD_ROOT=".."
```

Then you can deploy the executable using **west** again:

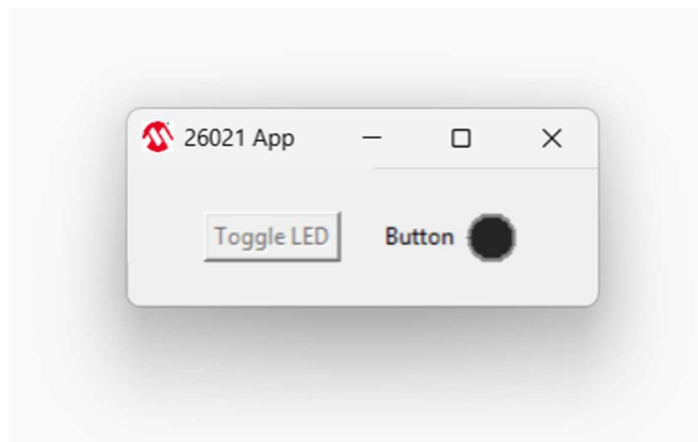
```
west flash
```

Connect the second USB cable, connected to **"TARGET USB"** port on the board, to an available USB port of the PC and launch the PC application from the VS Code terminal executing the following command block:

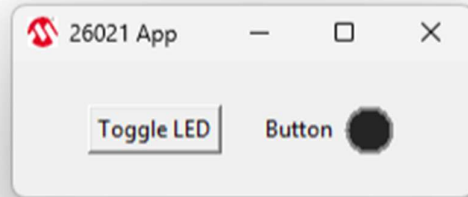
```
&{  
    cd .\App\  
    & '.\26021 App.exe'  
    cd ..  
}
```

Results

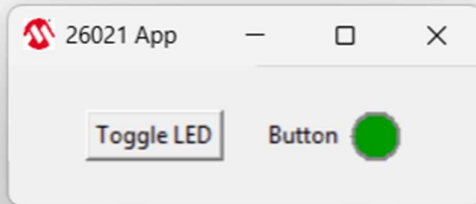
If the board is not connected through the "TARGET USB" port, the USB HID Device is not connected, and the app keep the "Toggle LED" button disabled:



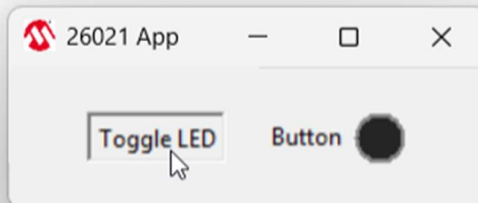
When you connect the board, the USB HID Device is enumerated and the app enables the "Toggle LED" button. If the button SW0 is not pressed, the button indicator is off (black):



If you press the SW0 button on the board, the button indicator is on (green):



If you press the "Toggle LED" button on the app, you toggle the status of LED0 on the board:



If you use the **Serial Monitor** Extension, opening the **Port** corresponding to the **EDBG** serial bridge at **115200** baud rate, you see the messages related to the status of the LED and the status of the button:



Summary

Congratulations! You have successfully configured the USB Device Stack to implement a **USB HID Device** and communicated with the PC application implementing a simple Command-Response protocol!

Copy & Paste

LAB 1

Step 1

LAB 2 - Redirect the Console to the USB

Step 1

```
/ {
    chosen {
        zephyr,console = &cdc_acm_uart;
    };
};

&zephyr_udc0 {
    cdc_acm_uart: cdc_acm_uart {
        compatible = "zephyr,cdc-acm-uart";
        label = "Zephyr USB CDC-ACM";
    };
};
```

Step 2

```
CONFIG_INPUT=y
CONFIG_USB_DEVICE_STACK_NEXT=y
CONFIG_USBD_CDC_ACM_CLASS=y
CONFIG_CDC_ACM_SERIAL_INITIALIZE_AT_BOOT=y
CONFIG_CDC_ACM_SERIAL_ENABLE_AT_BOOT=y
CONFIG_CDC_ACM_SERIAL_VID=0x04D8
CONFIG_CDC_ACM_SERIAL_PID=0x000A
CONFIG_CDC_ACM_SERIAL_MAX_POWER=250
CONFIG_CDC_ACM_SERIAL_MANUFACTURER_STRING="Microchip Technology Inc."
CONFIG_CDC_ACM_SERIAL_PRODUCT_STRING="Zephyr CDC-ACM Console"
CONFIG_CONSOLE_SUBSYS=y
CONFIG_CONSOLE_GETCHAR=y
```

Step 3

```
#include <zephyr/console/console.h>
#include <zephyr/input/input.h>

#define SW0_CODE DT_PROP(DT_NODELABEL(button0), zephyr_code)
```

Step 4

```
void input_cb(struct input_event *event, void *user_data)
{
    if (event->code == SW0_CODE) {
        if (event->value) {
            printk("Button pressed!\n");
        }
    }
    return;
}

INPUT_CALLBACK_DEFINE(NULL, input_cb, NULL);
```

Step 5

```
ret = console_init();
if (ret < 0)
{
    return 0;
}
printk("\nPress T on the keyboard to toggle the LED 0\n - or -\nPress SW0 on
the board to receive a message\n\n");
```

Step 6

Replace the **while(1)** loop inside the **main()** function with this one:

```
while (1) {
    unsigned char chr = console_getchar();
    if (chr == 'T' )
    {
        ret = gpio_pin_toggle_dt(&led);
        if (ret < 0) {
            return 0;
        }

        led_state = !led_state;
        printf("LED state: %s\n", led_state ? "ON" : "OFF");
        // k_msleep(SLEEP_TIME_MS);
    }
    else
    {
        printk("Received character: %c\n", chr);
    }
}
```

LAB 3 - Build a CDC device and communicate with a terminal on PC

Step 1

```
CONFIG_SERIAL=y
CONFIG_UART_INTERRUPT_DRIVEN=y
CONFIG_UART_LINE_CTRL=y
CONFIG_HWINFO=y
```

Step 2

- **usb.h**

```
#include <zephyr/usb/usbd.h>
#include <zephyr/drivers/uart.h>
extern const struct device *cdc_dev;
```

- **usb.c**

```
#include "usb.h"
const struct device *cdc_dev = DEVICE_DT_GET(DT_NODELABEL(cdc_acm_uart));
```

Step 3

```
USB_DEVICE_DEFINE(usbd_ctx, DEVICE_DT_GET(DT_NODELABEL(zephyr_udc0)), 0x04D8, 0x000A);
```

Step 4

```
USB_DESC_LANG_DEFINE(lang_desc);
USB_DESC_MANUFACTURER_DEFINE(mfr_desc, "Microchip Technology Inc.");
USB_DESC_PRODUCT_DEFINE(prod_desc, "Zephyr CDC-ACM");
USB_DESC_SERIAL_NUMBER_DEFINE(sn_desc);
```

Step 5

```
USB_DESC_CONFIG_DEFINE(fs_cfg_desc, "FS Configuration");
USB_CONFIGURATION_DEFINE(fs_config, USB_SCD_SELF_POWERED, 50, &fs_cfg_desc);
```

Step 6

```
void usb_msg_callback(struct usbd_context *const ctx, const struct usbd_msg *const msg)
{
    if (usbd_can_detect_vbus(ctx)) {
        switch (msg->type)
        {
            case USB_MSG_VBUS_READY:
                manage_usb_device(true);
                break;
            case USB_MSG_VBUS_REMOVED:
                manage_usb_device(false);
                break;
            default:
                break;
        }
    }
}
```

Step 7

- **usb.h**

```
extern bool managed_vbus;
int init_usb_device(void);
```

- **usb.c**

```
bool managed_vbus;
int init_usb_device(void)
{

}
```

Step 8

```
int err = 0;
err = usbd_add_descriptor(&usbd_ctx, &lang_desc);
if (err) {
    return err;
}
err = usbd_add_descriptor(&usbd_ctx, &mfr_desc);
if (err) {
    return err;
}
err = usbd_add_descriptor(&usbd_ctx, &prod_desc);
if (err) {
    return err;
}
err = usbd_add_descriptor(&usbd_ctx, &sn_desc);
if (err) {
    return err;
}
```

Step 9

```
err = usbd_add_configuration(&usbd_ctx, USB_SPEED_FS, &fs_config);
if (err) {
    return err;
}
```

Step 10

```
err = usbd_register_class(&usbd_ctx, "cdc_acm_0", USB_SPEED_FS, 1);
if (err) {
    return err;
}
```

Step 11

```
err = usbd_msg_register_cb(&usbd_ctx, usb_msg_callback);
if (err) {
    return err;
}
```

Step 12

```
err = usbd_init(&usbd_ctx);
if (err) {
    return err;
}
```

Step 13

```
managed_vbus = usbd_can_detect_vbus(&usbd_ctx);
return err;
```

Step 14

- **usb.c**

```
void manage_usb_device(bool connected)
{

}
```

- **usb.h**

```
void manage_usb_device(bool connected);
```

Step 15

```
typedef struct {
    uint8_t buf[128];
    uint16_t length;
} vcp_buffer;
extern vcp_buffer data_to_send;
```

Step 16

```
if (connected) {
    usbd_enable(&usbd_ctx);
    uint32_t dtr = 0;
    while (!dtr) {
        uart_line_ctrl_get(cdc_dev, UART_LINE_CTRL_DTR, &dtr);
        k_sleep(K_MSEC(100));
    }
    data_to_send.length = sprintf(data_to_send.buf, "\nPress T on the
keyboard to toggle the LED 0\n - or -\nPress SW0 on the board to receive a
message\n\n");
    uart_irq_tx_enable(cdc_dev);
}
else {
    usbd_disable(&usbd_ctx);
}
```

Step 17

```
#include "usb.h"
```

Step 18

```
vcp_buffer data_to_send;
vcp_buffer received_data;
```

Step 19

```
        data_to_send.length = sprintf(data_to_send.buf, "Button
pressed!\n\r");
        uart_irq_tx_enable(cdc_dev);
```

Step 20

```
void uart_irq_cb(const struct device *dev, void *user_data)
{
    static bool led_state = true;
    while (uart_irq_update(dev) && uart_irq_is_pending(dev)) {

    }
}
```

Step 21

```
    if (uart_irq_rx_ready(dev)) {
        received_data.length = uart_fifo_read(dev, received_data.buf,
sizeof(received_data.buf));
        if (received_data.length > 0) {
            if (received_data.buf[0] == 'T') {
                gpio_pin_toggle_dt(&led);
                led_state = !led_state;
                data_to_send.length = sprintf(data_to_send.buf, "LED state:
%s\n\r", led_state ? "ON" : "OFF");
            }
            else {
                data_to_send.length = sprintf(data_to_send.buf, "Received
character: %c\n\r", received_data.buf[0]);
            }
            uart_irq_tx_enable(dev);
        }
    }
```

Step 22

```
    if (uart_irq_tx_ready(dev)) {
        uart_fifo_fill(dev, data_to_send.buf, data_to_send.length);
        uart_irq_tx_disable(dev);
    }
```

Step 23

```
    if (!device_is_ready(cdc_dev)) {
        return 0;
    }
    ret = init_usb_device();
    if (ret < 0) {
        return 0;
    }
```

Step 24

```
    uart_irq_callback_set(cdc_dev, uart_irq_cb);
    uart_irq_rx_enable(cdc_dev);
```

Step 25

- Before ***input_cb()*** function:

```
#define VBUS_CODE DT_PROP(DT_NODELABEL(vbus0), zephyr_code)
```

- Inside ***input_cb()*** function:

```
if (!managed_vbus) {
    if (event->code == VBUS_CODE) {
        if (event->value) {
            manage_usb_device(true);
        }
        else {
            manage_usb_device(false);
        }
    }
}
```

Step 26

- Before ***main()*** function:

```
static const struct gpio_dt_spec vbus_gpio =
GPIO_DT_SPEC_GET(DT_NODELABEL(vbus0), gpios);
```

- Inside ***main()*** function:

```
if (!managed_vbus) {
    if (gpio_pin_get_dt(&vbus_gpio)) {
        manage_usb_device(true);
    }
}
```

Step 27

- Replace the target sources line with this one:

```
target_sources(app PRIVATE src/main.c src/usb.c)
```

LAB 4 - Implement a simple Command-Response protocol

Step 1

```
hid0: hid0 {
    compatible = "zephyr,hid-device";
    protocol-code = "none";
    in-report-size = <64>;
    out-report-size = <64>;
    in-polling-period-us = <1000>;
    out-polling-period-us = <1000>;
};
```

Step 2

```
CONFIG_USBD_HID_SUPPORT=y
```

Step 3

- **usb.h**

```
#include <zephyr/usb/class/usbd_hid.h>
extern const struct device *hid_dev;
```

- **usb.c**

```
const struct device *hid_dev = DEVICE_DT_GET(DT_NODELABEL(hid0));
```

Step 4

```
USBD_DEVICE_DEFINE(usbd_ctx, DEVICE_DT_GET(DT_NODELABEL(zephyr_udc0)), 0x04D8,
0x003F);
```

Step 5

```
USBD_DESC_PRODUCT_DEFINE(prod_desc, "Zephyr vendor HID");
```

Step 6

```
const uint8_t hid_report_descriptor[] = {
    0x06, 0x00, 0xFF,          // Usage Page (Vendor Defined 0xFF00)
    0x09, 0x01,                // Usage (Vendor Usage 1)
    0xA1, 0x01,                // Collection (Application)
    0x15, 0x00,                //   Logical Minimum (0)
    0x25, 0xFF,                //   Logical Maximum (255)
    0x75, 0x08,                //   Report Size (8 bits/byte)
    0x95, 0x40,                //   Report Count (64 bytes)
    0x09, 0x01,                // Usage (Vendor Usage 1)
    0x81, 0x02,                // Input (Data, Variable, Absolute)
    0x15, 0x00,                //   Logical Minimum (0)
    0x25, 0xFF,                //   Logical Maximum (255)
    0x75, 0x08,                //   Report Size (8 bits/byte)
    0x95, 0x40,                //   Report Count (64 bytes)
    0x09, 0x01,                // Usage (Vendor Usage 1)
    0x91, 0x02,                // Output (Data, Variable, Absolute)
    0xC0                       // End Collection
};
```


Step 7

- **usb.h**

```
int hid_get_report_cb(const struct device *dev, const uint8_t type, const
uint8_t id, const uint16_t len, uint8_t *const buf);
int hid_set_report_cb(const struct device *dev, const uint8_t type, const
uint8_t id, const uint16_t len, const uint8_t *const buf);
void hid_output_report_cb(const struct device *dev, const uint16_t len, const
uint8_t *const buf);
void hid_input_report_done_cb(const struct device *dev, const uint8_t *const
report);
```

- **usb.c**

```
struct hid_device_ops hid_ops = {
    .get_report = hid_get_report_cb,
    .set_report = hid_set_report_cb,
    .input_report_done = hid_input_report_done_cb,
    .output_report = hid_output_report_cb,
};
```

Step 8

```
err = usbd_register_class(&usbd_ctx, "hid_0", USB_SPEED_FS, 1);
if (err) {
    return err;
}
err = hid_device_register(hid_dev, hid_report_descriptor,
sizeof(hid_report_descriptor), &hid_ops);
if (err) {
    return err;
}
err = usbd_device_set_code_triple(&usbd_ctx, USB_SPEED_FS, 0, 0, 0);
if (err) {
    return err;
}
```

Step 9

```
if (!device_is_ready(hid_dev)) {
    return 0;
}
```

Step 10

```
static const struct gpio_dt_spec button =
GPIO_DT_SPEC_GET(DT_NODELABEL(button0), gpios);
```

Step 11

```
if (!gpio_is_ready_dt(&button)) {
    return 0;
}
ret = gpio_pin_configure_dt(&button, GPIO_INPUT);
if (ret < 0) {
    return 0;
}
```

Step 12

```
int hid_get_report_cb(const struct device *dev, const uint8_t type, const
uint8_t id, const uint16_t len, uint8_t *const buf)
{
    return -ENOTSUP;
}

int hid_set_report_cb(const struct device *dev, const uint8_t type, const
uint8_t id, const uint16_t len, const uint8_t *const buf)
{
    return -ENOTSUP;
}
```

Step 13

```
bool led_state = true;
void hid_output_report_cb(const struct device *dev, const uint16_t len, const
uint8_t *const buf)
{
    static uint8_t input_report[64];
    if (len > 0 && buf != NULL) {
        switch (buf[0]) {
            case 0x80: {

                break;
            }
            case 0x81: {

                break;
            }
        }
    }
}
```

Step 14

```
if(gpio_pin_toggle_dt(&led) != 0) {
    return;
}
led_state = !led_state;
printf("LED state: %s\n", led_state ? "ON" : "OFF");
```

Step 15

```
input_report[0] = buf[0];
if (gpio_pin_get_dt(&button)) {
    input_report[1] = 0;
}
else {
    input_report[1] = 1;
}
hid_device_submit_report(dev, 64, input_report);
```

Step 16

```
void hid_input_report_done_cb(const struct device *dev, const uint8_t *const
report)
{
    printk("LED state: %s\n", led_state ? "ON " : "OFF");
    printk("Button is %s\n\e[F\e[F", report[1] ? "not pressed" : "pressed    ");
}
```